

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 6

Roll no:06 PRN No: 12413462 Name of Student: Purva Patil

Subject Name : Deep Learning

Problem Statement :

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm :

1. Import the required Python libraries.
2. Load the Tomato or Soybean disease image dataset and organize the images according to their disease classes.
3. Explore the dataset by examining the number of images, image dimensions, and disease classes.
4. Visualize sample images from each disease class to understand the dataset.
5. Preprocess the images by resizing them to a fixed size and converting them into a suitable numerical format.
6. Normalize the pixel values to a range of 0 to 1.
7. Split the dataset into training and testing sets for model development and evaluation.
8. Create a CNN model consisting of convolutional layers, pooling layers, and fully connected Dense layers.
9. Compile the CNN model using an appropriate optimizer, loss function, and evaluation metric such as accuracy.
10. Train the CNN model using the training images and their corresponding disease labels.
11. Validate the model using validation data during training.
12. Predict the disease classes for the test images using the trained CNN model.
13. Evaluate the model performance using accuracy, loss, confusion matrix, and classification report.
14. Visualize the training and validation accuracy and loss using graphs.
15. Visualize the confusion matrix to analyze correctly and incorrectly classified disease classes.
16. Analyze the results and identify the performance of the CNN for tomato or soybean disease classification.

Code :

```
# -*- coding: utf-8 -*-  
"""Assignment_6_DL  
  
Automatically generated by Colab.  
  
Original file is located at  
    https://colab.research.google.com/drive/1TC9Qx7ZDgr6NUP1tmc-06xJAlamnr5GJ  
"""
```

```

!pip install -q kagglehub tensorflow matplotlib numpy

import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import kagglehub

print("TensorFlow version:", tf.__version__)
print("KaggleHub version:", kagglehub.__version__)

dataset_path = kagglehub.dataset_download(
    "mgmitiesh/potato-disease-detection-dataset"
)

print("Dataset downloaded to:")
print(dataset_path)

for root, dirs, files in os.walk(dataset_path):

    image_files = [
        f for f in files
        if f.lower().endswith((".jpg", ".jpeg", ".png"))
    ]

    if image_files:
        print(root)
        print("Images:", len(image_files))

class_folders = [
    "Potato__Early_blight",
    "Potato__Late_blight",
    "Potato__healthy"
]

data_dir = None

for root, dirs, files in os.walk(dataset_path):

    if all(folder in dirs for folder in class_folders):
        data_dir = root
        break

print("Dataset directory:")
print(data_dir)

IMG_SIZE = (128, 128)
BATCH_SIZE = 32
SEED = 123

```

```

EPOCHS = 20

print("Image size:", IMG_SIZE)
print("Batch size:", BATCH_SIZE)
print("Epochs:", EPOCHS)

train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(128, 128),
    batch_size=32
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(128, 128),
    batch_size=32
)

class_names = train_ds.class_names

print("Classes:")

for i, name in enumerate(class_names):
    print(i, "→", name)

print("\nNumber of classes:", len(class_names))

plt.figure(figsize=(10, 10))

for images, labels in train_ds.take(1):
    for i in range(9):
        plt.subplot(3, 3, i + 1)

        plt.imshow(images[i].numpy().astype("uint8"))

        plt.title(class_names[labels[i]])

        plt.axis("off")

plt.tight_layout()
plt.show()

```

```
AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.cache().shuffle(1000).prefetch(
    buffer_size=AUTOTUNE
)

val_ds = val_ds.cache().prefetch(
    buffer_size=AUTOTUNE
)

print("AUTOTUNE enabled successfully.")

data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1),
    tf.keras.layers.RandomContrast(0.1)
])

print("Data augmentation ready.")

model = tf.keras.Sequential([

    tf.keras.layers.Input(
        shape=(128, 128, 3)
    ),

    data_augmentation,

    tf.keras.layers.Rescaling(1.0 / 255),

    tf.keras.layers.Conv2D(
        32,
        (3, 3),
        activation="relu"
    ),

    tf.keras.layers.MaxPooling2D(
        (2, 2)
    ),

    tf.keras.layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    tf.keras.layers.MaxPooling2D(
```

```

        (2, 2)
    ),

    tf.keras.layers.Conv2D(
        128,
        (3, 3),
        activation="relu"
    ),

    tf.keras.layers.MaxPooling2D(
        (2, 2)
    ),

    tf.keras.layers.Flatten(),

    tf.keras.layers.Dense(
        128,
        activation="relu"
    ),

    tf.keras.layers.Dropout(0.5),

    tf.keras.layers.Dense(
        len(class_names),
        activation="softmax"
    )
])

model.summary()

model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=4,
    restore_best_weights=True
)

reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.2,
    patience=2,

```

```
        min_lr=0.00001
    )

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=[
        early_stopping,
        reduce_lr
    ]
)

loss, accuracy = model.evaluate(val_ds)

print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")

plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)
```

```

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")

plt.legend()
plt.grid()
plt.show()

from google.colab import files

uploaded = files.upload()

image_path = list(uploaded.keys())[0]

img = tf.keras.utils.load_img(
    image_path,
    target_size=IMG_SIZE
)

img_array = tf.keras.utils.img_to_array(img)

img_array = tf.expand_dims(
    img_array,
    axis=0
)

predictions = model.predict(img_array)

predicted_index = np.argmax(predictions[0])

predicted_class = class_names[predicted_index]

confidence = predictions[0][predicted_index] * 100

print("Predicted Class:", predicted_class)
print("Confidence:", round(confidence, 2), "%")

model.save("/content/potato_disease_cnn.keras")

print("Model saved successfully!")

```

Output :

Potato__Late_blight



Potato__Late_blight



Potato__Late_blight



Potato__Early_blight



Potato__Early_blight



Potato__Early_blight



Potato__Early_blight



Potato__Early_blight



Potato__Late_blight



Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 128, 128, 3)	0
rescaling_1 (Rescaling)	(None, 128, 128, 3)	0
conv2d_3 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_4 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_5 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_1 (Flatten)	(None, 25088)	0
dense_2 (Dense)	(None, 128)	3,211,392
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 3)	387

Total params: 3,305,027 (12.61 MB)





